

Assignment-12.2

Name : K.Tejaswini

Ht.no:2303A51425

Batch.No:21

TaskDescription-1(DataStructures–StackImplementationwithAI Assistance)

Prompt:

Generate a Python program that implements a Stack data structure using a class.Include the methodpush(element),pop(),peek(),andis_empty().Add propererrorhandlingforstackunderflowandincludecleardocstringsforeach method explaining their functionality. Take input from user.

Code:

```

class Stack:
    def is_empty(self):
        """
        True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0

# Example usage
if __name__ == "__main__":
    stack = Stack()
    while True:
        print("\nStack Operations:")
        print("1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Check if Empty")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

        if choice == '1':
            element = input("Enter the element to push: ")
            stack.push(element)
            print(f'{element}' has been pushed to the stack.")
        elif choice == '2':
            try:
                popped_element = stack.pop()
                print(f'{popped_element}' has been popped from the stack.")
            except IndexError as e:
                print(e)
        elif choice == '3':
            try:
                top_element = stack.peek()
                print(f"The top element is: '{top_element}'")
            except IndexError as e:
                print(e)
        elif choice == '4':
            if stack.is_empty():
                print("The stack is empty.")
            else:
                print("The stack is not empty.")
        elif choice == '5':
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 5.")

```

Output:

```

(base) PS C:\Users\WINDOMS\OneDrive\Desktop\Teja 3_2\AI Assit coding> & C:\Users\WINDOMS\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/WINDOMS/OneDrive/Desktop/Teja 3_2/AI Assit coding/lab.py"
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the element to push: 10
'10' has been pushed to the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 2
'10' has been popped from the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 3
Stack underflow: Cannot peek from an empty stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 4
The stack is empty.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 5
Exiting the program.
(base) PS C:\Users\WINDOMS\OneDrive\Desktop\Teja 3_2\AI Assit coding>

```

Justification:

A **Stack** is used because it follows the **Last In First Out (LIFO)** principle, which is suitable for operations where the most recent element must be accessed first.

Stacks allow efficient insertion and deletion operations using **push** and **pop** in constant time **O(1)**.

Other data structures like **queues** or **lists** do not strictly follow the LIFO order required for stack-based operations.

TaskDescription-2(Algorithms–LinearvsBinarySearchAnalysis) Prompt:

GeneratePythoncodetoimplementLinearSearchandBinarySearchwith functions:

`linear_search(arr,target)`and`binary_search(arr,target)`.

IncludeProperdocstringsexplainingtheworkingprinciple.Testthealgorithms with different input sizes (from user) and print the results for comparison.

Code:

```

def linear_search(arr, target):
    """
    Perform a linear search for the target in the given array.

    Parameters:
    arr (list): The list of elements to search through.
    target: The element to search for.

    Returns:
    int: The index of the target if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

def binary_search(arr, target):
    """
    Perform a binary search for the target in the given sorted array.

    Parameters:
    arr (list): The sorted list of elements to search through.
    target: The element to search for.

    Returns:
    int: The index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def main():
    arr=input("Enter a list of numbers (comma separated): ").split(',')
    target=int(input("Enter the target number to search for: "))
    linear_result = linear_search(arr, target)
    if linear_result != -1:
        print(f"Linear Search: Target found at index {linear_result}")
    else:
        print("Linear Search: Target not found")
    arr.sort() # Ensure the array is sorted for binary search
    binary_result = binary_search(arr, target)
    if binary_result != -1:
        print(f"Binary Search: Target found at index {binary_result}")
    else:
        print("Binary Search: Target not found")

if __name__ == "__main__":
    main()

```

Output:

```
Enter a list of numbers (comma separated): 10,20,55,77,89
Enter the target number to search for: 77
Linear Search: Target not found
```

Justification:

LinearSearchandBinarySearchusethe**Array/Listdatastructure**becauseit allows easy sequential and indexed access to elements.

BinarySearchspecificallyrequiresa **sortedarray**toefficientlydividethese search space.

Otherdatastructureslike**linkedlists****sortrees**arenotsuitablebecausetheydo not support efficient direct indexing required for Binary Search.

TaskDescription-3(TestDrivenDevelopment–SimpleCalculatorFunction)

Prompt:

GenerateunittestcasesfirstforasimplecalculatorinPythonusingunittest.

Thecalculatorshouldsupporttwooperations:additionandsubtraction.Take input from user for the numbers and the operation to perform.

Createaseparatetestfilenamedtest_calculator.py. Do not implement the calculator functions yet.

Code:

```

import unittest
class TestCalculator(unittest.TestCase):
    def test_addition(self):
        # Test addition of two numbers
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(0, 0), 0)

    def test_subtraction(self):
        # Test subtraction of two numbers
        self.assertEqual(subtract(5, 2), 3)
        self.assertEqual(subtract(1, -1), 2)
        self.assertEqual(subtract(0, 0), 0)
if __name__ == '__main__':
    unittest.main()

```

Now implement the calculator functions `add()` and `subtract()` in `calculator.py` so that all the existing unit tests pass.

```
import unittest
from calculator import add, subtract

class TestCalculator(unittest.TestCase):

    def test_add_positive(self):
        self.assertEqual(add(5, 3), 8)

    def test_add_negative(self):
        self.assertEqual(add(-2, -3), -5)

    def test_add_zero(self):
        self.assertEqual(add(0, 5), 5)

    def test_subtract_positive(self):
        self.assertEqual(subtract(10, 4), 6)

    def test_subtract_negative(self):
        self.assertEqual(subtract(-5, -2), -3)

    def test_subtract_zero(self):
        self.assertEqual(subtract(5, 0), 5)

if __name__ == "__main__":
    unittest.main()
```

Justification:

Test Driven Development ensures that test cases are written before implementing the actual functionality. It helps verify that the program meets requirements and reduces bugs during development. Using AI tools like Copilot speeds up writing tests and implementation while maintaining code correctness.

TaskDescription-4(DataStructures–QueueImplementationwithAI Assistance)

Prompt:

Generate a Python program that implements a Queue data structure using a class.

Include methods `enqueue(element)`, `dequeue()`, `front()`, and `is_empty()`.

Handle queue overflow and underflow conditions and include proper docstrings for each method.

Take input from user to perform queue operations.

Code:

```

def main():
    while True:
        print("\nQueue Operations:")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Front")
        print("4. Check if Empty")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            element = input("Enter the element to enqueue: ")
            try:
                queue.enqueue(element)
                print(f"Enqueued: {element}")
            except OverflowError as e:
                print(e)

        elif choice == 2:
            try:
                dequeued_element = queue.dequeue()
                print(f"Dequeued: {dequeued_element}")
            except IndexError as e:
                print(e)

        elif choice == 3:
            try:
                front_element = queue.front()
                print(f"Front element: {front_element}")
            except IndexError as e:
                print(e)

        elif choice == 4:
            if queue.is_empty():
                print("The queue is empty.")
            else:
                print("The queue is not empty.")

        elif choice == 5:
            print("Exiting the program.")
            break

        else:
            print("Invalid choice. Please try again.")

```

Output:

```
b.py"
Enter the capacity of the queue: 4

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice: 1
Enter the element to enqueue: 10
Enqueued: 10

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice: 2
Dequeued: 10

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice: 3
Queue underflow: Cannot access front element, the queue is empty.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice: 4
The queue is empty.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice: 5
Exiting the program.
```

Justification:

Queue is suitable because it follows the **FIFO (First In First Out)** principle, which is ideal for processing elements in the order they arrive. It is commonly used in **task scheduling, buffering, and request handling systems**. Other data structures like **Stack (LIFO)** or **Linked List** do not naturally maintain FIFO order for processing elements.

Task Description-5(Algorithms–Bubble Sort vs Selection Sort) Prompt:

Write Python functions `bubble_sort(arr)` and `selection_sort(arr)` with comments explaining each step.

Included docstrings mentioning time complexity and space complexity.

Return the sorted array and demonstrate the functions with sample test cases. Take input from user for the array to be sorted.

Code:

```

def bubble_sort(arr):
    """
    Sorts an array using the bubble sort algorithm.

    Time Complexity:  $O(n^2)$  in the worst and average case,  $O(n)$  in the best case (when the array is already sorted).
    Space Complexity:  $O(1)$  - Bubble sort is an in-place sorting algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: The sorted list.
    """
    n = len(arr)

    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr

def selection_sort(arr):
    """
    Sorts an array using the selection sort algorithm.
    Time Complexity:  $O(n^2)$  in all cases (best, average, and worst).
    Space Complexity:  $O(1)$  - Selection sort is an in-place sorting algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: The sorted list.
    """
    n = len(arr)

    # Traverse through all elements in the array
    for i in range(n):
        # Find the minimum element in the unsorted portion of the array
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first element of the unsorted portion
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Demonstrating the functions with sample test cases
if __name__ == "__main__":
    # Taking input from the user for the array to be sorted
    user_input = input("Enter the elements of the array to be sorted, separated by spaces: ")
    arr = list(map(int, user_input.split()))

    print("Original array:", arr)

    # Sorting using bubble sort
    sorted_arr_bubble = bubble_sort(arr.copy())
    print("Sorted array using bubble sort:", sorted_arr_bubble)

    # Sorting using selection sort
    sorted_arr_selection = selection_sort(arr.copy())
    print("Sorted array using selection sort:", sorted_arr_selection)

```

Output:

```
Enter a list of elements to sort (comma separated): 5,2,9,1,5,6  
Original array: ['5', '2', '9', '1', '5', '6']  
Sorted array using Bubble Sort: ['1', '2', '5', '5', '6', '9']  
Sorted array using Selection Sort: ['1', '2', '5', '5', '6', '9']
```

Justification:

Sorting algorithms like **Bubble Sort** and **Selection Sort** require indexed access, which is efficiently supported by arrays (Python lists). Arrays allow **easy swapping of elements using indices**, which is essential for these algorithms.

Other data structures like **linked lists, stacks, or queues** are **inefficient for direct element comparison and swapping**, making arrays the most suitable choice.